

D1 Detailed System Design

1. Header, Scope, and Conventions

Project Title: Ad-Vocates

Team Members: Dylan Shackleford, Max Marsh

Goal: Create a reliable and efficient tool that improves browsing performance while giving users greater privacy, security, and control over their web experience.

This D1 document provides a more detailed design for the **Filter Request**, **Local Storage**, and **Settings** components from D0. The **Dashboard**, **Statistic Recorder**, **Filter Updater**, and **Filter Validator** components will be detailed further in D2.

Figure Conventions

The D1 figures use the following conventions:

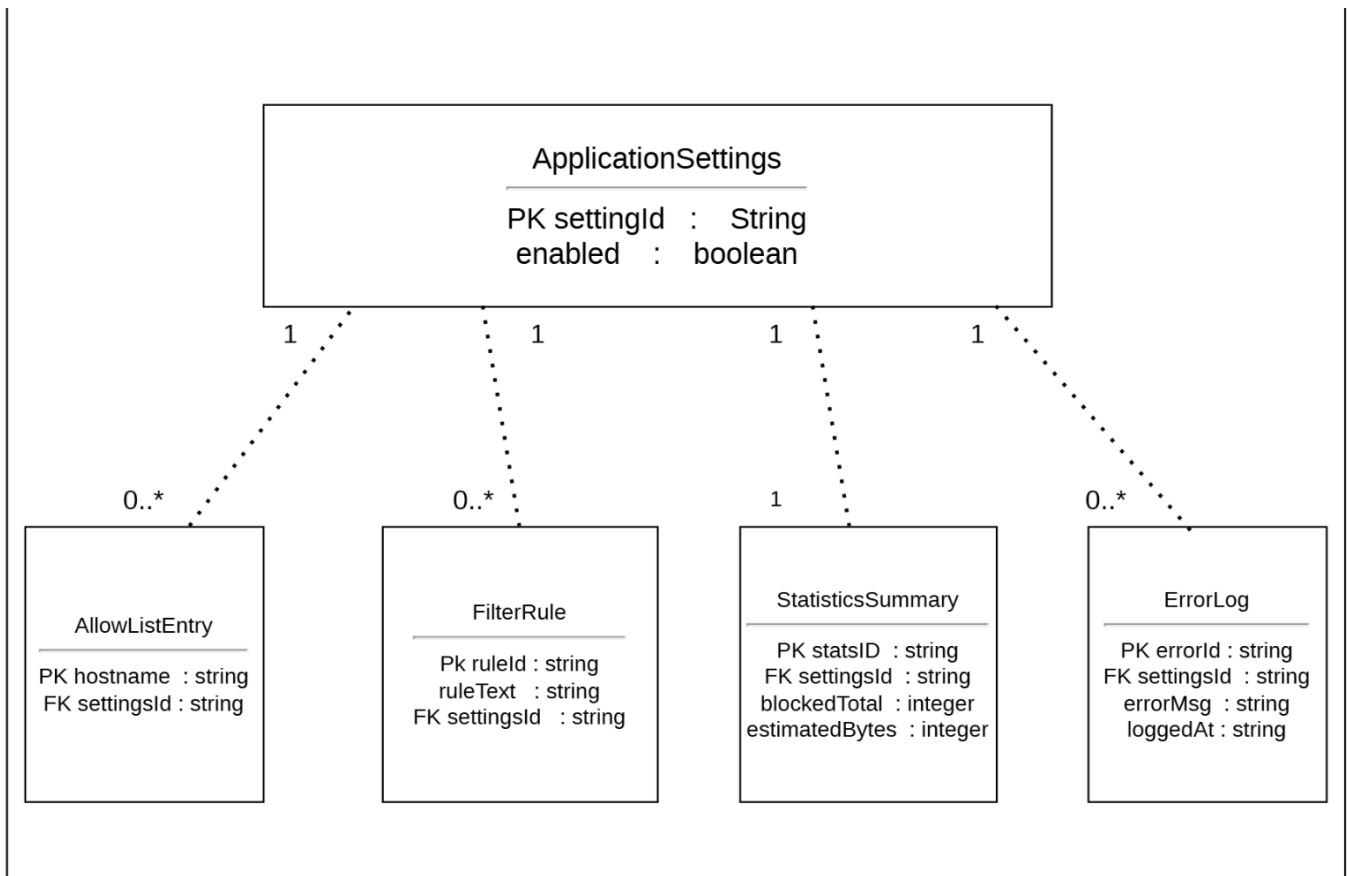
- Rectangular boxes represent system components or stored data entities.
- Lines between entities represent relationships between stored data.
- Arrows between system components represent the direction of communication or data flow.
- Primary keys are marked with **PK**.
- Foreign keys are marked with **FK**.
- Relationship cardinality is shown using **1:1** notation.
- Labels on arrows describe the type of data being passed between components.
- Components already defined in D0 keep the same names in D1 so the two designs can be compared directly.

2. Data Model, the D1 Diagram

The main data model for Ad-Vocates is contained within the **Local Storage** component from D0. Since Ad-Vocates runs locally as a browser extension, the project does not require a traditional server-side relational database. The stored information is small, belongs to one local installation of the extension, and does not need complex joins between tables.

For this reason, Ad-Vocates will use a **local object-based storage model** rather than a relational database. The model can be implemented using browser-local storage such as IndexedDB or another browser storage API. This keeps browsing-related information on the user's device and follows the D0 decision that filtering and application data should remain local.

The core stored records are shown below.



Data Entities

ApplicationSettings

Stores the main protection setting for the local Ad-Vocates installation.

Attribute	Type	Description
settingsId	string, PK	Unique identifier for the settings record
enabled	boolean	Indicates whether Ad-Vocates protection is enabled

There will normally be one **ApplicationSettings** record for an installation.

AllowlistEntry

Stores websites that the user has chosen to allow without normal filtering.

Attribute	Type	Description
hostname	string, PK	Normalized hostname of the allowed website
settingsId	string, FK	Connects the entry to the application settings

One ApplicationSettings record can have zero or many AllowlistEntry records.

This supports **US-03** and **US-05**, which require users to allow trusted sites or recover when blocking prevents an important website feature from working.

FilterRule

Stores the active filtering rules used by the Filter Request component.

Attribute	Type	Description
ruleId	string, PK	Unique identifier for the filter rule
ruleText	string	The filter rule or pattern that will be evaluated
settingsId	string, FK	Associates the rule with the active application settings

One ApplicationSettings record can have zero or many FilterRule records.

This supports **US-01** and **US-04**, because the rules are used to identify requests that should be blocked and can be replaced or updated as new filter data becomes available.

StatisticsSummary

Stores the current blocking totals used by the dashboard.

Attribute	Type	Description
statsId	string, PK	Unique identifier for the statistics record
blockedTotal	integer	Total number of blocked requests
estimatedBytes	integer	Estimated number of bytes prevented from loading

The application will normally maintain one current `StatisticsSummary` record rather than storing the user's complete browsing history.

This supports **US-02** and follows the D0 decision to store totals instead of individual browsing-history records.

ErrorLog

Stores errors that need to be recorded when filtering or local data operations fail.

Attribute	Type	Description
errorId	string, PK	Unique identifier for the error
errorMsg	string	Description of the error
loggedAt	string	Time the error was recorded in ISO 8601 format

Multiple `ErrorLog` records may exist. These records are intended for application errors and debugging and do not contain a user's browsing history.

Relationships and Cardinality

The main relationships in the D1 data model are:

- `ApplicationSettings` (1) to `AllowlistEntry` (0..*)
- `ApplicationSettings` (1) to `FilterRule` (0..*)
- `StatisticsSummary` is maintained as one local summary record.
- `ErrorLog` may contain zero or many independent error records.

The allowlist and filter rules are represented as separate records instead of being stored as one large value because individual entries may need to be added, removed, checked, or updated without replacing unrelated data.

Structural Decisions

Decision	Reason
----------	--------

AllowlistEntry is an entity instead of an attribute containing one large list	Each hostname needs to be checked, added, or removed independently. This directly supports US-03 and US-05.
FilterRule is an entity instead of one large filter string	Individual rules need to be loaded, validated, updated, and checked during filtering. This supports US-01 and US-04.
ApplicationSettings has a one-to-many relationship with AllowlistEntry	One local installation has one active set of settings but may contain many allowed websites.
ApplicationSettings has a one-to-many relationship with FilterRule	One active configuration may use many filtering rules at the same time.
Statistics are stored as a summary instead of individual browsing records	The dashboard needs blocked totals and estimated bytes, but storing full browsing history would collect unnecessary user information.
A local object store is used instead of a relational database	The application runs locally, has a small data model, does not require complex joins, and should avoid requiring a remote database or server.

Indexing Decisions

hostname is the primary key for AllowlistEntry so that the system can quickly determine whether the current website is allowlisted before filtering requests.

ruleId is the primary key for FilterRule so that duplicate rules can be identified and individual rules can be replaced or removed during filter updates. The actual request-matching process will use the loaded rule data rather than performing a database lookup for every browser request.

statsId does not require an additional secondary index because only one current statistics summary is expected.

If error records are retained, loggedAt may be indexed so recent errors can be retrieved in time order during debugging.

3. Core Algorithms

The two main algorithms for Ad-Vocates are request filtering and allowlist checking. These are important because if either one is too slow or does not work correctly, the blocker could miss unwanted requests or block something that should be allowed.

Algorithm 1: Request Filtering and Rule Matching

What it does

The request filtering algorithm decides whether a browser request should be blocked or allowed.

When the browser tries to load a resource, Ad-Vocates checks the request against the active filtering rules. If a matching blocking rule is found, the request is blocked. If no rule matches, the request is allowed to continue.

This algorithm supports **US-01**, which requires known advertising and tracking requests to be blocked.

Inputs

Input	Type	Description
requestUrl	string	Full URL of the requested resource
pageHost	string	Hostname of the page making the request
resourceType	string	Type of resource being requested, such as script, image, or frame
enabled	boolean	Whether Ad-Vocates protection is currently enabled
activeRules	FilterRule[]	Collection of active filtering rules

Each FilterRule contains:

ruleId: string
ruleText: string

Output

The algorithm returns:

decision: "ALLOW" | "BLOCK"

matchedRuleId: string | null

If the request is blocked, matchedRuleId identifies the rule that caused the block. If the request is allowed, matchedRuleId is null.

Expected Complexity

The active filter rules will be loaded into memory so that Local Storage does not need to be accessed for every request.

For the first version of the project, the rules can be grouped by domain or another useful identifier so Ad-Vocates only checks rules that could apply to the current request.

The expected lookup is approximately $O(1 + k)$, where k is the number of candidate rules that need to be checked for the requested domain.

For example, if Ad-Vocates contains around **10,000 active rules**, only a small subset of those rules should normally be checked for one request.

If the rule set increased by 100 times to around **1,000,000 rules**, checking every rule for every request would not be practical. The rules would need to stay indexed or grouped so the number of rules checked for one request stays relatively small.

This matters because the filtering algorithm may run many times while a single webpage is loading.

Why This Approach Was Chosen

The team chose to load and organize filter rules in memory instead of reading every rule from Local Storage for every request.

A simple linear scan through every active rule would be easier to implement, but it would take approximately $O(n)$ time for every request, where n is the total number of filter rules. This would become slower as the filter list grows.

Organizing the rules before request processing takes some extra setup, but it allows the Filter Request component to make faster decisions during normal browsing.

Edge Cases

- If no filtering rules are loaded, the request is allowed.
- If protection is disabled, the request is allowed without checking the rules.
- Duplicate rules should not cause the blocked request counter to increase more than once for the same request.
- If multiple rules match, the request is still only blocked once.

- If a rule cannot be processed, it should be skipped instead of stopping the entire filtering process.
- If the request URL cannot be parsed, Ad-Vocates should not crash.
- If no rule matches, the request is allowed.

Algorithm 2: Allowlist Checking

What it does

Before normal filtering is applied, Ad-Vocates checks whether the current website has been added to the allowlist.

If the website is allowlisted, normal blocking rules are skipped for that site. If the website is not allowlisted, request filtering continues normally.

This algorithm supports **US-03** and **US-05**, because users need a way to trust a website or recover if blocking causes an important website feature to stop working.

Inputs

Input	Type	Description
pageHost	string	Hostname of the website currently being visited
allowlist	Set<string>	Set of normalized hostnames that the user has allowed

Output

The algorithm returns:

isAllowlisted: boolean

If the value is true, normal filtering should be skipped for the website.

If the value is false, Ad-Vocates continues with the normal filtering process.

Expected Complexity

The allowlist will be stored in a set-like structure so checking whether a hostname exists has an expected complexity of $O(1)$.

For a normal installation, the allowlist is expected to contain fewer than around **100 websites**.

Even if the allowlist grew by 100 times to around **10,000 websites**, an average $O(1)$ lookup should still have very little effect on browsing performance.

Why This Approach Was Chosen

A set or hash-based lookup was chosen because the system only needs to determine whether a hostname is currently allowlisted.

An alternative would be storing the allowlist as an array and checking each hostname one at a time. That would require an $O(n)$ search and would become less efficient as the allowlist grows.

A set also helps prevent duplicate hostname entries.

Edge Cases

- If the allowlist is empty, the hostname is treated as not allowlisted.
- Duplicate hostnames should not be stored.
- Hostnames should be converted to lowercase before comparison.
- Hostnames should be normalized before lookup.
- Subdomains should be treated separately unless the team later decides to support parent-domain matching.
- Invalid or empty hostnames should not be added to the allowlist.
- If a hostname is removed from the allowlist, normal filtering should resume for that site.

4. Build-versus-Reuse Decisions

Ad-Vocates will use a combination of components developed by the team and existing browser technologies. Existing tools will be reused when they provide well-tested functionality that does not need to be recreated specifically for this project. The team will focus its development work on the filtering, settings, statistics, and user-control features that are specific to Ad-Vocates.

Component or Feature	Build or Reuse	Library, API, or Service	License	Reason
Request filtering logic	Build	Ad-Vocates filtering logic	Project code	The rules for deciding whether a request should be blocked are a core part of the project and need to work with our settings, allowlist, and statistics.

Browser request handling	Reuse	Chromium Extension APIs	Chromium/browser platform APIs	Browser APIs already provide supported methods for interacting with browser requests. Reusing them is more reliable than attempting to build browser-level request handling ourselves.
Filter lists	Reuse	Publicly available ad and tracker filter lists	Depends on selected filter list	Existing filter lists provide mature collections of known advertising and tracking rules. The specific list and its license will be verified before it is included in the project.
Filter rule organization and matching	Build	Ad-Vocates filtering engine	Project code	The team needs control over how rules are loaded, organized, and checked so filtering performance can be measured and improved.
Allowlist logic	Build	Ad-Vocates settings logic	Project code	Allowlisting is directly tied to the project's settings and filtering behavior and is simple enough to implement within the application.
Local data storage	Reuse	Browser storage API / IndexedDB	Browser platform API	Browser storage is mature, local to the user's device, and avoids the need to build or host a separate database service.
Settings management	Build	Ad-Vocates settings component	Project code	The settings behavior is specific to Ad-Vocates and needs to control protection status, filtering, and allowlist behavior.
Dashboard interface	Build	HTML, CSS, and JavaScript/TypeScript	Project code	The dashboard is specific to the information and controls provided by Ad-Vocates and will be designed by the team.

Basic URL and hostname parsing	Reuse	Browser URL API	Web platform API	URL parsing is standardized and already provided by the browser. Reusing it reduces errors and avoids creating custom URL parsing code.
Sorting and basic collection operations	Reuse	Built-in JavaScript/TypeScript functionality	ECMAScript/Web platform	Standard language functions are mature and sufficient for basic collection operations, so there is no reason to implement custom sorting or collection utilities.

Reuse Evaluation

Libraries and services considered for Ad-Vocates will be checked for maturity, licensing, performance, and fit before they are added to the project.

Browser-provided APIs are preferred when they already provide the required functionality because they are widely supported, maintained as part of the browser platform, and do not require additional hosting.

For external filter lists, the team will verify the license and usage requirements of the specific list before including it in the final application. The team will also consider the size and performance impact of the list because filtering rules may be checked many times while websites are loading.

Third-party libraries will only be added when they provide a clear advantage over built-in browser or JavaScript functionality.

Possible Filter Lists

The team has not selected a final filter list yet. The following are possible options that could be evaluated for Ad-Vocates based on licensing, maturity, performance, and compatibility.

- **EasyList**

A widely used filter list focused mainly on blocking advertisements. It is commonly used by many ad-blocking projects and would be a strong candidate for the main advertising filter.

- **EasyPrivacy**

A companion list focused more on trackers, analytics, and other privacy-related requests. This could be used alongside an advertising-focused list.

- **AdGuard Base Filter**

A maintained filter list designed to block advertisements on English-language websites. AdGuard's filter repository is actively maintained and its filters are also used by other blocking software.

- **AdGuard Tracking Protection Filter**

A filter focused on privacy-related tracking requests. This could be evaluated as an alternative to EasyPrivacy or as part of a combined filtering approach.

Before choosing a final list, we will compare the license, number of rules, update frequency, browser compatibility, and performance impact of each option.

5. API Contract

Ad-Vocates will mainly use internal methods and browser-extension messaging instead of a traditional web API. These methods define how the Settings, Filter Request, Local Storage, and related components exchange data.

Method 1: evaluateRequest

Determines whether a browser request should be allowed or blocked.

Field	Type	Required	Valid Range / Description
requestUrl	string	Yes	Valid HTTP or HTTPS URL
pageHost	string	Yes	Valid normalized hostname
resourceType	string	Yes	Browser resource type such as image, script, frame, or stylesheet
requestId	string	Yes	Unique request identifier

Output

```
{
  decision: "ALLOW" | "BLOCK",
  matchedRuleId: string | null
}
```

decision indicates whether the request should continue.

matchedRuleId contains the matching filter rule identifier if the request is blocked. Otherwise, it is null.

Errors

Error Code	When It Happens
INVALID_URL	The request URL cannot be parsed
FILTERS_UNAVAILABLE	No valid filter data is currently available
INTERNAL_ERROR	An unexpected filtering error occurs

Method 2: isAllowlisted

Checks whether the current website is included in the user's allowlist.

Field	Type	Required	Valid Range / Description
hostname	string	Yes	Valid normalized website hostname

Output

```
{
  isAllowlisted: boolean
}
```

Errors

Error Code	When It Happens
INVALID_HOSTNAME	The hostname is empty or invalid
STORAGE_ERROR	The allowlist cannot be read from local storage

Method 3: updateSettings

Updates the main protection settings for Ad-Vocates.

Field	Type	Required	Valid Range / Description
enabled	boolean	Yes	true or false

allowlist	string[]	No	List of valid normalized hostnames
-----------	----------	----	------------------------------------

Output

```
{
  success: boolean,
  updatedCount: integer
}
```

updatedCount represents the number of stored setting values or allowlist entries changed.

Errors

Error Code	When It Happens
INVALID_SETTING	A provided value is outside the accepted type or range
INVALID_HOSTNAME	One of the supplied allowlist entries is invalid
STORAGE_ERROR	The updated settings cannot be saved

Method 4: getStatistics

Returns the current blocking statistics stored by Ad-Vocates.

Inputs

No parameters are required.

Output

```
{
  blockedTotal: integer,
  estimatedBytes: integer
}
```

blockedTotal is the number of requests blocked.

estimatedBytes is the estimated number of bytes prevented from loading.

Errors

Error Code	When It Happens
STORAGE_ERROR	Statistics cannot be read from local storage

Method 5: refreshFilters

Requests a new version of the selected external filter list and updates the active rules if the new data is valid.

Inputs

No user-supplied parameters are required.

Output

```
{
  success: boolean,
  ruleCount: integer,
  updatedAt: string
}
```

ruleCount is the number of valid filter rules loaded.

updatedAt is an ISO 8601 timestamp showing when the filter data was last updated.

Errors

Error Code	When It Happens
NETWORK_ERROR	The filter list cannot be downloaded
INVALID_FILTER_DATA	The downloaded filter data cannot be validated
STORAGE_ERROR	The validated filter rules cannot be saved

Example Request and Response

Example request sent to the Filter Request component:

```
{
  "method": "evaluateRequest",
  "requestUrl": "https://example-ad-domain.com/banner.js",
  "pageHost": "example.com",
  "resourceType": "script",
  "requestId": "req-1042"
}
```

Example response:

```
{
  "decision": "BLOCK",
  "matchedRuleId": "rule-125"
}
```

Versioning

Internal Ad-Vocates component interfaces will use a version number such as v1. A breaking change is any change that removes a field, changes a field's type or meaning, changes a method name, or requires an existing caller to change how it sends or receives data. Breaking changes will require a new interface version.

6. Technology Choices with Justification

Several technology choices are still being finalized. For the major parts of the system, we are currently considering two realistic options instead of committing to technologies before development and testing begin.

Programming Language

The two main options are **JavaScript** and **TypeScript**. Both work directly with modern browser extension APIs and have strong community support. JavaScript would have the lowest setup complexity and would be easier to begin developing with immediately. TypeScript would add static type checking, which could help catch mistakes in filter rules, settings objects, and internal API messages before runtime. Both are free and open technologies and should provide enough performance for the planned request filtering logic. Neither requires paid hosting because the extension will initially run locally in the user's browser. The final choice will depend mainly on which language allows the team to develop and debug the extension most effectively.

Browser Platform

The project can either target **Chrome and other Chromium-based browsers first** or use the broader **WebExtensions approach** with the goal of supporting both Chromium and Firefox. Chromium-first development would reduce the number of browser differences the team has to test during the first version and provides access to the current Chrome extension APIs. A broader WebExtensions approach would make the application available to more users but would require additional compatibility testing. Both options have large developer communities, mature documentation, and no direct licensing or hosting cost. For the first working version, Chromium is likely to require less development effort, while broader browser support can be considered after the main features are stable.

Front End

The dashboard can be built using **plain HTML, CSS, and JavaScript/TypeScript** or with a framework such as **React**. Plain HTML and CSS would keep the extension small and reduce the number of third-party dependencies. React provides reusable components and may make a larger interface easier to manage, but it adds additional build tools and dependencies. Both approaches have strong community support and are available without licensing cost. Since the Ad-Vocates dashboard is expected to be relatively small, plain HTML and CSS may provide better simplicity and performance, while React remains an option if the dashboard becomes more complex.

Local Data Storage

The two main storage options are the browser's **extension storage API** and **IndexedDB**. The extension storage API is simpler and works well for small settings such as whether protection is enabled and which websites are allowlisted. IndexedDB is better suited for larger amounts of structured data such as large filter lists or more detailed statistics. Both are built into modern browsers, have mature documentation, require no additional license, and do not create hosting costs. The final design may use the extension storage API for small settings while using IndexedDB if the selected filter list is too large for simple key-value storage.

Filter List

Ad-Vocates will reuse an **existing maintained filter list** rather than attempting to create a complete advertising and tracker list from scratch. The exact list has not yet been selected. The team will compare at least two candidates based on license compatibility, update frequency, maturity, list size, coverage, and performance. Reusing an established list allows the team to focus development time on the filtering engine, allowlist behavior, statistics, and user controls. Before a filter list is included in a downloadable release, its license will be checked to make sure redistribution is allowed.

Hosting and Distribution

The first version of Ad-Vocates is planned as a **local browser extension without a required cloud backend**. One option is to distribute development releases through GitHub, while another option is eventual distribution through a browser extension store such as the Chrome Web Store. Keeping filtering and user settings local reduces hosting cost and avoids requiring a remote server to store browsing-related information. GitHub provides a simple way to distribute source code and development releases, while an extension store would make installation and updates easier for general users. A hosted backend could be added later if a future feature requires centralized services, but it is not necessary for the current design.